

Tor Vergata University of Rome



TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

FACULTY OF ENGINEERING

**COMPUTER SCIENCE, CONTROL AND GEOINFORMATION
CYCLE XXXVII**

PhD Thesis

**A UNIFIED DISTRIBUTED SOFTWARE
ARCHITECTURE FOR AUTONOMOUS SYSTEMS**

ADVISOR

Prof. Sergio Galeani

CANDIDATE

Roberto Masocco

COORDINATOR

Prof. Francesco Quaglia

A.Y. 2023/2024



TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

COMPUTER SCIENCE, CONTROL AND GEOINFORMATION
CYCLE XXXVII

A UNIFIED DISTRIBUTED SOFTWARE ARCHITECTURE FOR AUTONOMOUS SYSTEMS

PHD THESIS

Advisor:

Prof. Sergio Galeani

Signature: _____

Coordinator:

Prof. Francesco Quaglia

Signature: _____

Candidate:

Roberto Masocco

Signature: _____

A.Y. 2023/2024

Tor Vergata University of Rome, Faculty of Engineering
Via del Politecnico 1, Rome, Italy

Wings are a constraint that makes
it possible to fly.
— Robert Bringhurst

To my parents...

Acknowledgements

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Lausanne, 12 Mars 2011

Preface

This is the summary of three years of work spanning multiple areas. From the problem to the solution. It is inevitable to name some tools and concepts before they are explained later on among chapters.

Footnote should be cited here Galeani, Masocco, and Sassano 2022; Galperti et al. 2024.

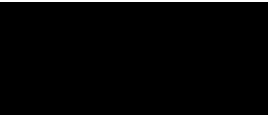
Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante.

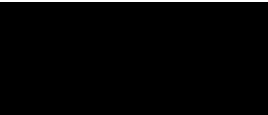
Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes,
nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis.
Pellentesque cursus luctus mauris.

Contents

Acknowledgements	i
Preface	ii
List of figures	v
List of tables	vi
1 Introduction	1
2 The importance of an architecture	6
2.1 Foreword	6
Bibliography	8



List of Figures



List of Tables

1 Introduction

In the history of humanity, many turning points have been marked by technological advancements. From the invention of the wheel to the steam engine, from the light bulb to the computer, technology has continuously changed how we live, work, and see the world. Its impact on society has always been profound and sometimes positive, sometimes negative. The changes it brought affected our daily habits, introducing new ways of doing things and sometimes even new ways of thinking. With the same spirit that drove our ancestors to develop *tools* made of stones and sticks to make their lives easier while harvesting food, we are now developing ever more powerful machines of all sorts to handle the challenges of the modern world, either in conjunction with a human operator or in complete autonomy. Moreover, considering the recent developments of artificial intelligence, these tools have reached such a level of sophistication that they may reshape our thought processes soon.

The technological advancements that brought us these new solutions increased not only their capabilities but also their complexity and, ultimately, their variety. An excellent example is the industrial manipulator robot: a machine that can perform various tasks, from simple pick-and-place operations to more complex ones, such as welding, painting, or even surgery. These machines have evolved from mechanical arms composed of multiple actuators, all connected to a single power and computational unit, to a network of sensors and microcontrollers, grouped in subsystems all

interconnected to a central unit, which is, in turn, responsible for their coordination, supervision, and control, as well as for interfacing with human operators. In this case, this evolution has been driven by the need to increase the robot's capabilities, making it more versatile and adaptable to different tasks, and by the need to make it more reliable and safe, reducing the risk of accidents and downtime due to maintenance. Another suitable example would be the modern airplane: considering even a small civilian aircraft, originally equipped with an engine, control surfaces, and a few instruments, it has evolved into a complex system of systems composed of a multitude of sensors, probes, indicators, and avionics. In this scenario, the evolution has been driven by specific needs still related to safety and comfort but ultimately summarisable in the need to ease the lives of both the passengers and the crew.

This design and development paradigm has recently become commonly denoted by the adjective *distributed*. The definitive characteristic of a distributed system is its decomposition into a network of subsystems. Each one is responsible for a specific task; all of them are interconnected, and they all perform a piece of the job that the entire macroscopic system has been designed to do. This decomposition allows for more efficient use of the resources, better scalability, and higher reliability, as the failure of a single subsystem does not necessarily lead to the failure of the whole system. Moreover, the distribution of the subsystems allows for a better organization of the system's functionalities, as each subsystem can be designed and developed independently from the others, following a modular approach that eases the task of understanding, controlling, and maintaining the system as a whole. This approach has been widely adopted in many fields, from industrial automation to aerospace, automotive to robotics, telecommunications to energy. Another peculiar characteristic of a distributed system is the possibility, in many contexts, to deploy its parts in different locations: coming back to the manipulator example, it is common to

have the sensors and the actuators distributed along the robot's arm and the control unit placed in a separate location, while in the airplane example, the sensors and the avionics are distributed along the fuselage and the wings, while the cockpit is placed in the front of the aircraft; in pure software systems, the subsystems can be deployed in different servers, data centers, or even continents, with the additional advantage of partitioning the workload generated by the various regions among local computational nodes. These degrees of physical distribution allow great flexibility in design and resource allocation, but they also introduce new challenges in communication, synchronization, and fault tolerance.

All of this innovation comes at a price: when the multitude of subsystems, subcomponents, and submodules increases, new problems arise. The system's complexity grows, and so does the difficulty of understanding, controlling, and maintaining it. The interactions between the different parts of the system become intricate. With strong dependencies among modules and no redundancy, the consequences of a failure in one of them can propagate to the others, causing a cascade of failures that can lead to catastrophic events. A deep understanding is needed to prevent events like these, for which a good organization of the system *architecture* is of paramount importance. Moreover, a well-designed architecture may also be incredibly beneficial in two other stages: the design and development of the system and its maintenance. In the former, a clear and well-structured architecture naturally leads to a clear and well-structured design, following a top-down approach that starts from the system's requirements and ends with the definition of the subsystems and their interfaces, each following a common set of standards and best practices that adhere to the structure of the architecture itself. In the latter, a well-structured architecture allows for a clear identification of the subsystems and their interactions, easing diagnosing and fixing a failure or an error. Finally, a good architecture can also be a powerful resource for the

evolution of the system as a whole, allowing for the addition of new functionalities or the replacement of old components with new ones without the need to redesign the whole system from scratch, significantly reducing the time and the costs of such an operation.

The subject of this work is the development of a software architecture for distributed systems intended to control dynamic processes and autonomous agents. This setting may seem specific, but it is pretty broad: all of the above examples fall into these categories, representing topics of high interest in today's research and industrial environments. All the design and development problems listed above still apply to these cases, which also bring their own specific challenges addressed in the following. The main goal of this work is to provide a unified solution to these problems by defining a software architecture that can be used as a reference for the design and development of distributed systems in the contexts mentioned above. The proposed architecture is designed to be modular and flexible, and it is based on a set of standards and best practices that may guide the development of the subsystems and their interactions. Moreover, the architecture is designed to be adaptable to and deployable on different hardware platforms, to provide an abstraction layer for solutions ranging from embedded systems to general-purpose computers, and to easily integrate other software modules like third-party libraries. Finally, the architecture is built to be easily usable by developers, providing a unified set of tools and utilities that ease the task of developing, testing, and deploying new modules for the classes of systems mentioned above. The end goal of this work comes from a fundamental necessity identified by the author during his experience as a researcher and developer in the field of automation and robotics: having a standard development environment suitable for a single developer or an entire team, capable of replicating almost entirely the system configuration of a target platform on a development machine and enabling distributed applications to

work and behave in the same way on all supported platforms, from build to run time. This way, the integration overhead gets dramatically cut, allowing developers to focus on their most essential and only true task: research and development, leading quickly to new technological advancements.

Just like our primordial ancestors faced the necessity of building tools to ease their lives, this work follows the same approach: identify the problem, lay down the requirements, design the solution, build it, test it, and finally use it. The requirements that drive the design of the architecture are identified and discussed in the following Chapters, and the proposed solutions are based on the author's experience in the field of automation and robotics, as well as on the state of the art in the field of software engineering and distributed systems.

2 The importance of an architecture

2.1 Foreword

The development of an autonomous system is a process that involves various stages. Each stage is typically characterized by defining and solving issues related to different aspects of the system, ranging from the tasks it will be required to perform, to its construction and programming, and even to how the data it collects and that affect its operation should be presented to potential operators. Generalizing from a specific application case, most issues can be classified into a few high-level categories. Although it may seem counterintuitive from the division that follows, these categories are not always addressed sequentially, nor are they entirely independent of each other. To address situations like these, various project management techniques have been developed in recent times, aimed at minimizing the risk that an error made in an earlier phase could compromise subsequent stages, potentially undermining the work completed and extending development timeframes. In light of the observations made in Chapter 1, it is clear that, with the increasing complexity of modern automated systems and of the tasks they must perform, these topics are more relevant than ever. In particular, new tools made available to developers and designers will need to reflect these specificities, simplify the management of typical situations where possible, and highlight potential critical issues to aid in problem-solving. Practical experience

shows that, despite the variety of cases and applications, the challenges faced are almost always, if not identical, very similar, as are the strategies to address them.

The aim of this chapter is to briefly present the issues typically encountered when designing an autonomous system, highlighting their unique characteristics without focusing on a particular application area, and then to examine some recent hardware and software solutions that enable the optimization of design, development, and testing activities. The discussion starts at a relatively high level of abstraction but becomes progressively more specific, forming the guiding thread that connects all subsequent chapters, finally highlighting the necessity of a well-structured software architecture to address the issues raised.

Bibliography

Galeani, S., R. Masocco, and M. Sassano (Dec. 2022). “Dynamic control allocation for sampled-data closed-loop systems”. In: *Proc. of the IEEE 61st Conference on Decision and Control (CDC)*. IEEE, pp. 4206–4211.

Galperti, C. et al. (Nov. 2024). “Overview of the TCV digital real-time plasma control system and its applications”. In: *Fusion Engineering and Design* 208, p. 114640.